KAOP / AGENT OPERATIONS

YOUR TEAMS HAVE AGENTS.

WHO OWNS THE RISK?



Something useful? • Can we see how? • Within the limits?

Echizeni Ikpah | Product Engineer

Today: one real incident and a first step.

YOUR PRESENTER

Echizeni Ikpah

Product Engineer, Komodor

I help teams adopt KAOP:

- Get the first workflow running
- Measure the results with you
- Bring your feedback to Product

Please ask questions any time. This is a conversation.

01 **Your environment**
Where agent sprawl would hurt first

02 **Why KAOP**
One view, real limits, work that closes

03 **A live incident**
A real alert, the evidence, a human decision

04 **A reviewed fix**
A separate code reviewer on a real pull request

05 **Results and a first step**
What I measured, and a pilot that fits your team

Where would agent sprawl hurt your team first?

01 OWNERSHIP

WHO OWNS IT?

No inventory, no clear owner

02 ACCESS

WHAT CAN IT TOUCH?

Broad tokens, unclear access

03 EVIDENCE

WHAT ACTUALLY HAPPENED?

Scattered runs and no decision trail

DISCOVERY QUESTION **Which would be hardest for your team to manage as usage grows?**

Three outcomes. Not another agent dashboard.

01

ONE VIEW

Agents, owners, runs and cost

02

REAL LIMITS

Permissions enforced outside prompts

03

WORK THAT CLOSES

Alert → evidence → decision → review

Built on KAOP’s shared operating model

BUILD | RUN | GOVERN | OPTIMIZE

Two triggers. Two workflows. One operating model.

WORKFLOW A / GRAFANA ALERT

Alert
restart signal

→

Investigate
K8s + Grafana

→

Propose
remediation plan

→

Decide
person restores

WORKFLOW B / FRESH GITHUB PR

Person opens
one-line fix

→

Code reviewer
comments on PR

→

Human controls
merge + deploy

→

Verify
repeat the fault

The PR review does not auto-start from the incident. The two workflows have separate triggers and privileges.

A prompt is not a permission boundary.

AGENT	ALLOWED WORK	CANNOT DO	ENFORCED BY
Kubernetes RCA	Read pods, events, logs	Secrets; exec; modify	Kubernetes RBAC
Grafana investigator	Read metrics + alerts	Write / unlisted tools	KAOP gateway allow list
Remediation	Propose fix, undo, check	Touch a live system	No live-system tools
Code reviewer	Comment on PR	Approve or merge	A person merges

TESTED: Grafana tools outside the list refused • Kubernetes writes denied • Over-budget alert not investigated

Live proof: six things you should see.

01 **REAL SIGNAL**

Grafana restart alert

02 **EVIDENCE**

KAOP RCA and tool calls

03 **BOUNDARY**

Agent cannot write

04 **DECISION**

Person restores Redis

05 **REVIEW**

Fresh GitHub bot comment

06 **ONE VIEW**

Owner, cost and audit trail

THE ROOT CAUSE OF AVOIDABLE RESTARTS

Redis-sensitive /readyz was misused for liveness.

REDIS OUTAGE

Trigger, not probe defect

LIVE 01 | Signal → evidence-backed diagnosis

SCREEN 1

TERMINAL

Redis down
API restart + liveness 503

Kubernetes kills a healthy API because its liveness check needs Redis.

→

SCREEN 2

GRAFANA CLOUD

Alert fires
workload + timestamp

Fires about 3 minutes in. Says what and when, not why.

→

SCREEN 3

KAOP INCIDENT

RCA + Grafana
orchestrated finding

Two specialists. The orchestrator scores each possible cause.

REDIS OUTAGE = TRIGGER **DEPENDENCY-COUPLED LIVENESS = RESTART ROOT CAUSE**

LIVE 02 | Human decision → reviewed change

01	**KAOP proposes**	Restore Redis now + durable probe fix
02	**PERSON APPLIES**	kubectl restores Redis in the lab
03	**PERSON OPENS PR**	Change liveness /readyz → /healthz
04	**REVIEWER COMMENTS**	Independent KAOP GitHub workflow

NO AGENT DID THESE

- ✕ Changed production
- ✕ Approved or merged the PR
- ✕ Took credit for the fix

Fix tested on the same outage: 5 liveness kills in 5 minutes before, 0 after.

A completed run is not a correct run.

ANSWER QUALITY

1 of 5 → 10 of 10

Agent found the actual cause

Without / with one documented probe standard.
Same investigation input; small lab samples.

COST OF THE GRAFANA INVESTIGATOR

$0.68 → $0.53

Reported average per run

28 read tools versus a broader toolset.
Five runs per setup; identical cases vary.

NOT A BENCHMARK: Hosted agent changed version during tests; evaluation must be repeated in the customer environment.

Earn autonomy through measured adoption.



Score the workflow before increasing its authority.

- ✓ **Correct RCA + evidence**
- ✓ **Time to useful decision**
- ✓ **Policy compliance**
- ✓ **Cost per useful run**
- ✓ **Engineer adoption**

One pilot. One owner. One decision to expand.

THE FIRST WORKFLOW

- Named technical owner + interested users
- One alert or PR workflow, not a fleet migration
- Scoped access, human gates, baseline metrics
- Review actual outputs and adoption after pilot

MY QUESTION

Which workflow would you want to prove first?

Who would be the right person to help us scope it?

FINAL THOUGHT

MAKE AGENTS ACCOUNTABLE

BEFORE MAKING THEM AUTONOMOUS.

ONE VIEW | ENFORCED LIMITS | EVIDENCE → DECISION

What would that first pilot need to prove for you?

Lab results: what was actually measured

Fault vs fix	Bad probe: 5 liveness kills in 5 min; corrected probe: zero.
Incident timing	Alert at +3:26 and +3:09 in two October 25 rehearsals.
Decision timing	Remediation suggestions ready around +11:30 and +13:36.
PR reviewer	Automated comment in 35 s and 33 s on rehearsal PRs.
Quality	10/10 with standards doc; 1/5 without, in stated comparison.
Cost	About $0.68 → $0.53 average for 5 vs 5 Grafana agent runs.

Caveats: Single-node kind lab, small samples, shared dependency cause, and a hosted-agent upgrade during experiments.

Who had authority in this specific lab

IDENTITY / ACTION	ENFORCED SCOPE	EXCLUDED ACTIONS
Kubernetes RCA	Kubernetes read-only via ServiceAccount RBAC	Secrets, exec, delete, patch, scale
Grafana investigator	28 selected read tools through KAOP gateway	Tool families outside allow list
Workflow budget	$40 a day; new runs blocked at the cap	Runs past the cap
Remediation	Plan and recommend, with verification	No tools on any live system
GitHub reviewer	Issue-style review comment on a real PR	Cannot approve or merge

The human applies recovery; the workflow records that distinction.